



Fig. 4: Conversion of array using MPFS utility program

page outgoing packets.

More sub-routines used in the source code are listed in Table III.

Since all the communications are done through a computer network line, IP address and Web page are required. In order to see the HTML Web page properly, your browser should support AJAX.

IP address. The IP address changes from one computer to another. The dynamic IP address, which is assigned by DHCP client routine, is eliminated and fixed IP address 192.168.0.80 is used in this project. To find out the IP address of a system, you can use ipconfig command at the command prompt. There are many software modules written for this project including IP, ARP and ICMP. The IP, ARP and ICMP protocol implementations are ported from Microchip TCP/IP stack only. But the necessary codes for TCP protocol and HTTP are self-written in C and compiled using AVR-GCC compiler to fit into the memory size of ATmega128. The TCP protocol implementation and HTTP routines are available in http.c source file.

HTML page. The Web pages are developed using Aurora Web Editor 2008 Professional and converted into arrays of data using MPFS utility program from Microchip. Then the converted file is used in the firmware to access the Web page. The detailed steps for this conversion are out of the scope of this article. Only some details are given here. First, you need to put all required files into one folder, say kat. The MPFS program creates arrays for all files in the folder and the generated arrays is available in the user-defined k.h header file. Fig. 4 shows the invoke

command and the arrays information in k.h file. In this typical example, the folder k in D drive contains Web page files. The command is:

```
D:\>mpfs d:\k k.h /c
```

The generated k.h file has to be edited to make it suitable for use.

In original header file, array names are given as MPFS_0000, MPFS_0001 and MPFS_0003, etc. But here, we are using array names with two alphabets like IN and LA instead of MPFS_000, etc for easy decoding. In this project, the k.h file is MPFSimg.h. The content of the MPFSimg.h starts with #include 'GenericTypeDefs.h' line. The structure definition of the MPFSimg.h file is as shown below:

```
#include "GenericTypeDefs.h"

typedef struct __FAT_TABLE_ENTRY
{
    int8_t Flags;
    ROM uint8_t* Address;
    uint8_t Name[12];
    uint16_t size;
} FAT_TABLE_ENTRY;
```

At the end of the MPFSimg.h file are ROM table entry details containing different array names. The modified table information is shown below for reference:

```
ROM FAT_TABLE_ENTRY MPFS_Start[] =
{
    { 0x00, IN, 'I', 'N', '.', 'H', 'T',
      'M', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, EE, 'E', 'E', '.', 'H', 'T',
        'M', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, IB, 'I', 'B', '.', 'J', 'P',
        'G', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, IC, 'I', 'C', '.', 'J', 'P',
        'G', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, LB, 'L', 'B', '.', 'J', 'P',
        'G', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, LA, 'L', 'A', '.', 'J', 'P',
        'G', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, LC, 'L', 'C', '.', 'J', 'P',
        'G', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
      { 0x00, LD, 'L', 'D', '.', 'J', 'P',
        'G', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
```

```
{ 0x00, LE, 'L', 'E', '.', 'J', 'P',
  'G', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
  { 0x00, ST, 'S', 'T', '.', 'C', 'G',
    'I', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
    { 0x04, 0xffffffff, 'E', 'N', 'D',
      ' ', 'O', ' ', 'F', ' ', 'E', 'A', 'T',
      ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
```

```
};
```

The array name definition should be edited and changed as shown below to store it in the program memory of ATmega128. The key tag PROGMEM is used to store the array in program memory:

```
static uint8_t PROGMEM ST[] =
{
    0x3c, 0x21, 0x44, 0x4f, 0x43, 0x54,
    0x59, 0x50, 0x45, 0x20, 0x48, 0x54,
    0x4d, 0x4c, 0x20, 0x50,
    0x55, 0x42, 0x4c, 0x49, 0x43, 0x20,
    0x22, 0x2d, 0x2f, 0x2f, 0x57, 0x33,
    0x43, 0x2f, 0x2f, 0x44,
    0x54, 0x44, 0x20, 0x48, 0x54, 0x4d,
    0x4c, 0x20, 0x34, 0x2e, 0x30, 0x20,
    0x54, 0x72, 0x61, 0x6e,
};
```

A typical example code for sending Web page information array to network unit is appended below:

```
else if (senddata==0x4542) {
    ne_prints("HTTP/1.1 200 OK\r\n");
    ne_prints("Content-type:text/html\r\n");
    send_file(&EB, sizeof(EB)-4, remote, yy);
}
```

The send_file routine is used to send the array information and 'ne_prints' is used to send HTML header information to the Web page, so that you can see the water-level status on your PC monitor through microcontroller.

LCD driver routines are available in lcd_lib.c and the definitions in lcd_lib.h files, respectively. All the relevant files are included in the EFY DVD.

Programming the code. A 6-pin header is provided in the circuit for programming ATmega128 using ISP protocol. Before programming the ATmega128, you need to remember two things: First ensure that ENC28J60 is removed from the board, otherwise,